

基于 Magic Sets 的并行时序推理方法与实现 结题答辩汇报

报告人：赵楷越 韦东良

2025年7月3日

饮水思源 · 爱国荣校



1

项目背景

2

成员工作与成果

3

项目总结与心得

01

项目背景





项目背景



流数据广泛存在于金融交易、网络异常检测、设备预警、智能制造等领域。**基于流数据的符号推理**为上述领域的众多应用提供了有力支撑。

A Simple DatalogMTL Example:

$$\text{Authorised}(x) \leftarrow \Diamond_{[0,2]} \text{NegativeLFT}(x) \wedge \Box_{[0,15]} \text{FullyVaccinated}(x).$$

处理流数据



金融交易

网络信息

制造设备

...

(更多时序信息)

DatalogMTL 是一种新兴的支持流数据推理的逻辑编程语言，相较于命令式编程，使用 DatalogMTL 规则语言能够更好地**固化专家知识，降低沟通和开发成本。**





项目特色与创新点



- ④ DatalogMTL 的计算复杂性较高，使得实际推理任务十分困难。为了解决这一难题，我们将 Magic Sets 技术引入 DatalogMTL。通过自下而上的推理，模拟**自上而下**的评估来提升效率。这是一种**全新的优化思路**。
- ④ Magic Sets 技术通过重写程序和数据，确保在给定查询下，重写后的程序与原始程序提供一致的答案，同时，能限制推理范围，避免冗余推理，减少不必要的计算开销和数据量，显著提高查询效率。尽管 Magic Sets 在 Datalog 语言中是一种成熟的优化方法，但在 DatalogMTL 中**尚未有对应的优化方法被发现与证明**。
- ④ 将 Magic Sets 方法应用于 DatalogMTL 推理中，需要克服 DatalogMTL 推理中**特有的难点问题**，如推理结果可能无限大、推理过程可能不终止等。
- ④ 针对现有 DatalogMTL 推理引擎（如基于 Python 开发的 MeTeoR）性能不足的问题，**采用 C++ 语言开发**高性能推理模块，提升处理大规模数据的效率。





项目进度总览



- 2023-11-14: 大创项目学校审核通过
- 2023-11 至 2024-2: 前期准备、理论学习、论文研读、初步熟悉工具与平台
- 2024-2 至 2024-3: 算法的初步形成、组会研讨
- 2024-4: 代码原型的实现
- 2024-4-7: 中期答辩
- 2024-4 至 2024-7: 算法的严谨证明、算法的进一步优化、在数据集上的性能测试
- 2024-8 至今: 论文的写作与投稿、rebuttal、引擎的性能优化



02

成员工作与成果





项目成果



- ④ **理论研究**: 研究 Magic Sets 技术的基本原理, 包括程序装饰、规则重写和查询优化。探索 Magic Sets 在 DatalogMTL 推理中的应用场景及其扩展性, 研究支持否定和聚合等复杂特性。
- ④ **算法设计**: 成功设计并实现了适用于 DatalogMTL 的 Magic Sets 规则重写方法, 解决了推理结果可能无限大、推理过程可能不终止等问题。
- ④ **系统实现**: 实现原型系统, 包括基于 C++ 实现 Magic Sets 规则重写模块以及对应的命令行界面工具, 并结合 MeTeoR 时序推理工具、支持 DatalogMTL 推理任务。
- ④ **实验评估**: 通过实验验证系统的性能, 分析其在不同数据集上的可扩展性和效率。结果表明, 与传统的 DatalogMTL 推理方法相比, 本方法在查询效率上具有显著优势, 特别是在大规模数据集上可有 2-100,000 倍的查询性能提升。
- ④ **论文写作**: 论文投稿被 AAAI 2025 会议 (属于 CCF-A 类会议) 接收, 并入选 AAAI Oral Presentation。





DatalogMTL 语言与 MeTeoR 推理器

MeTeoR: metric temporal reasoner

Examples:

Dataset:

```
RedList(adam)@[0,20]
HighRisk(david)@(-inf,+inf)
HighRisk(ernesto)@(-inf,+inf)
Transaction(adam,betty)@2.87
Transaction(betty,charlie)@12.15
Transaction(charlie,david)@17.5
Transaction(charlie,ernesto)@129.43
```

Query:

```
Suspect(david)@200
```

Program:

```
TransactionChain(X,Y) :- Transaction(X,Y), RedList(X)
TransactionChain(X,Z) :- SOMETIME[-24,0] TransactionChain(X,Y), Transaction(Y,Z)
ALWAYS[0,+inf] Suspect(Y) :- TransactionChain(X,Y), HighRisk(Y)
```

MeTeoR 推理器

$\mathfrak{I}, t \models \Diamond_{\varrho} M$ iff $\mathfrak{I}, t' \models M$ for some t' with $t - t' \in \varrho$
 $\mathfrak{I}, t \models \Diamond_{\varrho} M$ iff $\mathfrak{I}, t' \models M$ for some t' with $t' - t \in \varrho$
 $\mathfrak{I}, t \models \Box_{\varrho} M$ iff $\mathfrak{I}, t' \models M$ for all t' with $t - t' \in \varrho$
 $\mathfrak{I}, t \models \Box_{\varrho} M$ iff $\mathfrak{I}, t' \models M$ for all t' with $t' - t \in \varrho$
 $\mathfrak{I}, t \models M_1 \mathcal{S}_{\varrho} M_2$ iff $\mathfrak{I}, t' \models M_2$ for some t' with $t - t' \in \varrho$
 and $\mathfrak{I}, t'' \models M_1$ for all $t'' \in (t', t)$
 $\mathfrak{I}, t \models M_1 \mathcal{U}_{\varrho} M_2$ iff $\mathfrak{I}, t' \models M_2$ for some t' with $t' - t \in \varrho$
 and $\mathfrak{I}, t'' \models M_1$ for all $t'' \in (t, t')$

DatalogMTL 扩展运算符





原生 Datalog 语言中的 Magic Sets 方法

Input: A Datalog program $\mathcal{P}$, and a query $Q = g(\bar{c})$.
Output: The optimized program $MS(Q, \mathcal{P})$.
var S : stack of adorned predicates; $adornedRules$, $modifiedRules$, $magicRules$: set of rules;
begin
1. $modifiedRules := \emptyset$; $magicRules := BuildQuerySeeds(Q, S)$;
2. **while** $S \neq \emptyset$ **do**
3. $p^\alpha := S.pop()$;
4. **for each** rule $r \in \mathcal{P}$ with $H(r) = p(\bar{t}_p)$ **do**
5. $adornedRules := adornedRules \cup Adorn(r, p^\alpha, S)$;
6. **end while**
7. **for each** rule $r_a \in adornedRules$ **do**
8. $magicRules := magicRules \cup Generate(r_a)$;
9. **for each** rule $r_a \in adornedRules$ **do**
10. $modifiedRules := modifiedRules \cup \{Modify(r_a)\}$;
11. $MS(Q, \mathcal{P}) := magicRules \cup modifiedRules$;
12. **return** $MS(Q, \mathcal{P})$;
end.

Datalog 中 Magic Sets 算法的伪代码

Program

```
path(X, Y) :- edge(X, Y),  
path(X, Y) :- edge(X, Z), path(Z, Y).
```

Query

+ query path(1, 5)

Magic Sets Transformation (top-down)



Utilizing information from
Query to speed up reasoning

```
magic_pathbb(1, 5).  
magic_pathbb(Z, Y) :- magic_pathbb(X, Y), edge(X, Z).  
path(X, Y) :- magic_pathbb(X, Y), edge(X, Y).  
path(X, Y) :- magic_pathbb(X, Y), edge(X, Z), path(Z, Y).
```

Program After Magic Sets Method





原生 Datalog 语言中的 Magic Sets 方法



Program

```
path(X, Y) :- edge(X, Y),  
path(X, Y) :- edge(X, Z), path(Z, Y).
```

Query

+ query path(1, 5)

1 Adornment



```
path_bb(x,y) :- edge(x,y).  
path_bb(x,y) :- edge(x,z), path_bb(z,y).
```

2 Generation



```
magic_path_bb(1,5).  
magic_path_bb(z,y) :- magic_path_bb(x,y), edge(x,z).
```

During the generation process, some atoms enter the head from the body, and some atoms enter the body from the head.





原生 Datalog 语言中的 Magic Sets 方法

Program

```
path(X, Y) :- edge(X, Y),  
path(X, Y) :- edge(X, Z), path(Z, Y).
```

Query

+ query path(1, 5)

3 Modification



```
path(x,y) :- magic_path_bb(x,y), edge(x,y).  
path(x,y) :- magic_path_bb(x,y), edge(x,z), path(z,y).
```

During the modification process, some atoms enter the body from the head.

Final Program (2 Generation + 3 Modification)

```
magic_pathbb(1, 5).  
magic_pathbb(Z, Y) :- magic_pathbb(X, Y), edge(X, Z).  
path(X, Y) :- magic_pathbb(X, Y), edge(X, Y).  
path(X, Y) :- magic_pathbb(X, Y), edge(X, Z), path(Z, Y).
```





将 Magic Sets 方法运用到 DatalogMTL

探索：针对 metric atom 从 head 进入 body 与从 body 进入 head 这两种情况，对 metric atom 进行处理。使得规则能推理出正确的结果，同时符合规则语法的要求。

metric atom所带的 operator	Generation, atom从head 进入body时	Generation, atom从body 进入head时	Modification, atom从head 进入body时
ALWAYS	变为SOMETIME	保留ALWAYS不变	变为SOMETIME
SOMETIME	/	变为ALWAYS	/

原理：对时间限制的“放宽”

是否会导致多余的事实被推导出来？：不会，因为影响的其实只有 magic 事实，而 magic 事实只相当于起到过滤器的作用，把程序的 evaluation 限制在可能推导出查询结果的范围内。





算法的不断完善

$$M_1 S_{\rho} M_2 \rightarrow (BoxMinus)_{\rho} M_2 + (BoxMinus)_{[0,e]} M_1$$

$$M_1 U_{\rho} M_2 \rightarrow (BoxPlus)_{\rho} M_2 + (BoxPlus)_{[0,e]} M_1$$

例如 (NoSymptoms和Vaccinated均为EDB)

```
Immune(X) :- NoSymptoms(X) UNTIL[-28,-21] vaccinated(X)
```

(这是meteor网站自带的例子, UNTIL[-28,-21] 其实就是 SINCE[21,28])

在generation过程中, 变为

```
ALWAYS[-28,0] magic_NoSymptoms_b(X) :- magic_Immune_b(X)
ALWAYS[-28,-21] magic_Vaccinated_b(X) :- magic_Immune_b(X)
```

针对最复杂的 Since 与 Until 语义, 算法的变化

查看我们的算法推出来的规则集, 发现在2和4两条由generation这一步得到的规则中, 均有 memberOf_bf:

```
2 magic_a1:GoodDepartment_f:-magic_a1:SmartStudent_b(X), Diamondminus[1.000000,2.000000]a1:memberOf_bf(X,Y), a1:UndergraduateStudent(X)
3 Boxminus[1.000000,3.000000]magic_a1:memberOf_b(X):-magic_a1:SmartStudent_b(X)
4 magic_a1:GoodDepartment_f:-magic_a1:SmartStudent_b(X), Diamondminus[1.000000,3.000000]a1:memberOf_bf(X,Y), a1:GraduateStudent(X)
```

但是没有任何规则能推出memberOf_bf, 能推出的只有memberOf:

```
27 a1:SmartStudent(X):-magic_a1:SmartStudent_f, Diamondminus[1.000000,2.000000]a1:memberOf(X,Y), a1:memberOf(Y,X)
28 a1:memberOf(X,Y):-magic_a1:memberOf_ff, a1:memberOf(Y,X)
29 a1:memberOf(X,Y):-magic_a1:memberOf_ff, a1:worksFor(X,Y), a1:memberOf(Y,X)
30 a1:worksFor(X,Y):-magic_a1:worksFor_ff, a1:memberOf(X,Y)
```

28和29是modification得到的规则, 在这一步会strip off adornment, 将memberOf_bf变为memberOf。

针对 IDB 与 EDB 的各种组合情况, 算法的调整





DatalogMTL 中 Magic Sets 算法的伪代码



Algorithm 1: MagicRewriting

Input : A DatalogMTL program Π , dataset $\mathcal{D}$, and query $q = Q(t)@_{\varrho}$

Output: A DatalogMTL program Π' and dataset $\mathcal{D}'$

- 1 $\mathcal{D}' := \mathcal{D} \cup \{m_Q^{\gamma_0}(bt(t, \gamma_0))@_{\varrho}\}$;
- 2 $A := \{Q^{\gamma_0}\}$; $\Pi_a := \emptyset$;
- 3 Π_a := a program obtained from Π by applying Step 2 from Section “Magic Set Rewriting” until $A = \emptyset$;
- 4 $\Pi' := \emptyset$; $\Pi'_1 := \emptyset$; $\Pi'_2 := \emptyset$;
- 5 **foreach** rule $r \in \Pi_a$ **do**
- 6 Let $\square_{\varrho'} R^{\gamma}(t)$, with $\square \in \{\boxplus, \boxminus\}$, be the head of r ;
- 7 **if** $\square = \boxplus$ **then** $r' := r$ with $\boxplus_{\varrho'} m_R^{\gamma}(bv(t, \gamma))$ added as the first body atom ;
- 8 **if** $\square = \boxminus$ **then** $r' := r$ with $\boxminus_{\varrho'} m_R^{\gamma}(bv(t, \gamma))$ added as the first body atom ;
- 9 $\Pi'_1 := \Pi'_1 \cup \{r'\}$;
- 10 **foreach** rule $r \in \Pi'_1$ **do**
- 11 **foreach** $M \in \text{body}(r)$ with an idb predicate **do**
- 12 $H := \text{MagicHeadAtoms}(M)$;
- 13 **foreach** atom $M' \in H$ **do**
- 14 $r' := r$ with head replaced by M' and body modified by deleting M and all body atoms to the right of M ;
- 15 $\Pi'_2 := \Pi'_2 \cup \{r'\}$;
- 16 $\Pi' = \Pi'_1 \cup \Pi'_2$;
- 17 Delete adornments from non-magic predicates in Π' ;
- 18 **return** $(\Pi', \mathcal{D}')$;

Algorithm 1: Magic Sets Transformation

Algorithm 2: MagicHeadAtoms

Input : A metric atom M

Output: A set H of metric atoms with magic predicates

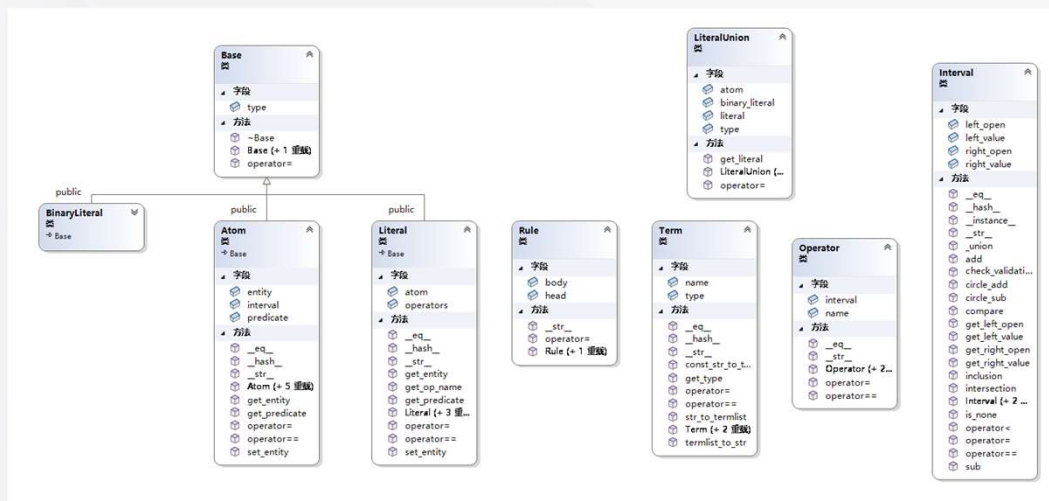
- 1 $H := \emptyset$; $M' := M$;
- 2 **foreach** adorned idb relational atom $R^{\gamma}(t)$ in M **do**
- 3 Replace $R^{\gamma}(t)$ in M' with $m_R^{\gamma}(bv(t, \gamma))$;
- 4 **if** $M' = \boxplus_{\varrho} M_1$ or $M' = \boxminus_{\varrho} M_1$ **then** $H := \{M'\}$;
- 5 **if** $M' = \boxplus_{\varrho} M_1$ **then** $H := \{\boxplus_{\varrho} M_1\}$;
- 6 **if** $M' = \boxminus_{\varrho} M_1$ **then** $H := \{\boxminus_{\varrho} M_1\}$;
- 7 **if** $M' = M_2 \mathcal{S}_{\varrho} M_1$ **then**
- 8 $t_{max} :=$ the right endpoint of ϱ ;
- 9 **if** M_2 contains a magic predicate **then**
- 10 $H := H \cup \{\boxminus_{[0, t_{max}]} M_2\}$;
- 11 **if** M_1 contains a magic predicate **then**
- 12 $H := H \cup \{\boxminus_{\varrho} M_1\}$;
- 13 **if** $M' = M_2 \mathcal{U}_{\varrho} M_1$ **then**
- 14 $t_{max} :=$ the right endpoint of ϱ ;
- 15 **if** M_2 contains a magic predicate **then**
- 16 $H := H \cup \{\boxplus_{[0, t_{max}]} M_2\}$;
- 17 **if** M_1 contains a magic predicate **then**
- 18 $H := H \cup \{\boxplus_{\varrho} M_1\}$;
- 19 **return** H ;

Algorithm 2: Metric Atom Modification

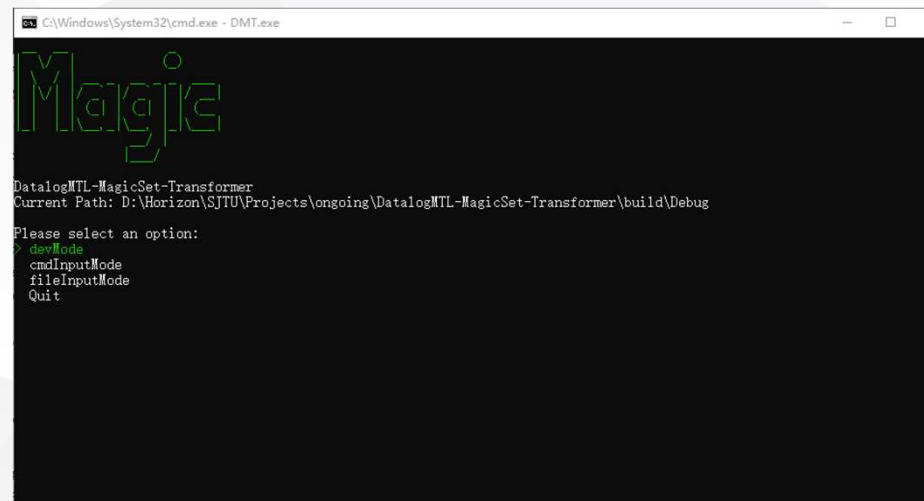




代码原型的实现



项目的类图



实际运行展示





性能测试

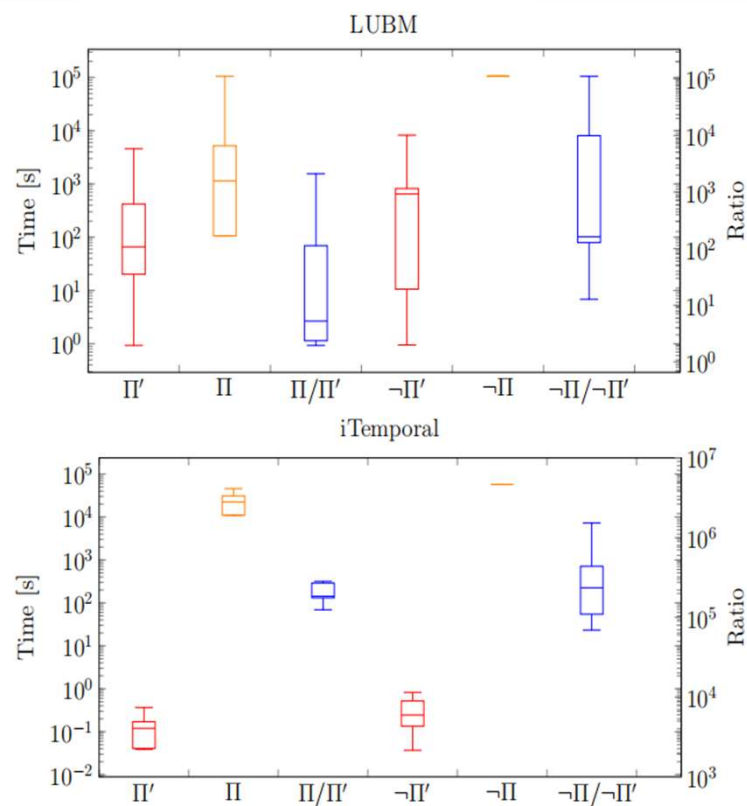


- 在实验部分、我们进行了对比实验以及可扩展性实验。
- 在**对比实验**中，我们在标准的 LUBMt、iTemporal 和气象数据集上，分别运行了 119、35 和 28 个查询，覆盖所有 IDB 谓词。结果表明，Magic Sets 重写方法在查询时间上**显著优于原始程序**。
- 随后、我们在 LUBMt 和 iTemporal 测试集上进行了**可扩展性实验**。对于 LUBMt，我们在 5 种规模的数据集 (10^2 – 10^6) 上每种规模运行 30 个查询；对于 iTemporal，在 10 – 10^5 规模的数据集上，每种规模运行 270 个查询。实验结果显示，Magic Sets 重写方法优化后、在不同数据集规模下均表现出**更优的查询执行效率**，平均执行时间显著低于原始程序。
- 综上实验，验证了 Magic Sets 在实际应用中的高效性和实用性。





性能测试



在各类大规模数据集上,
与 baseline 的对比

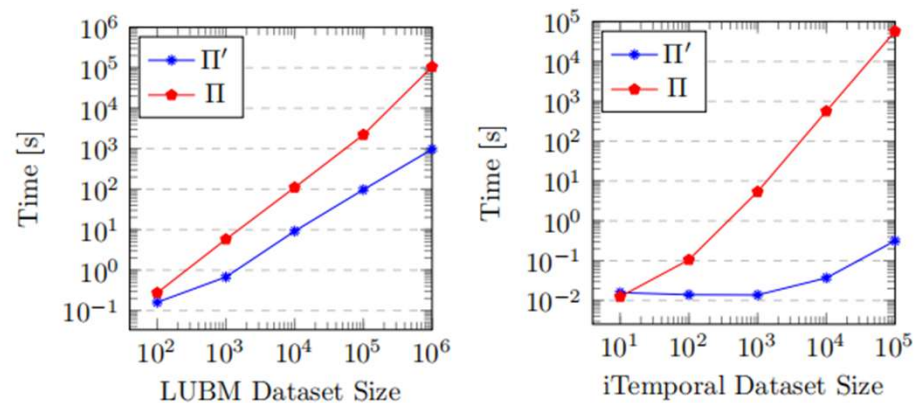


Figure 2: Scalability results, where Π' stands for a magic program and Π for an original program

可扩展性测试





论文成果

Goal-Driven Reasoning in DatalogMTL with Magic Sets

Shaoyu Wang^{1*}, Kaiyue Zhao^{1*}, Dongliang Wei^{1*},
Przemysław Andrzej Wałęga^{2,3}, Dingmin Wang², Hongming Cai¹, Pan Hu^{1†}

¹School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, China

²Department of Computer Science, University of Oxford, UK

³School of Electronic Engineering and Computer Science, Queen Mary University of London, UK
{royalrasins, Horizon52183, weidongliang9, hmcai, pan.hu}@sjtu.edu.cn, {przemyslaw.walega, dingmin.wang}@cs.ox.ac.uk

Abstract

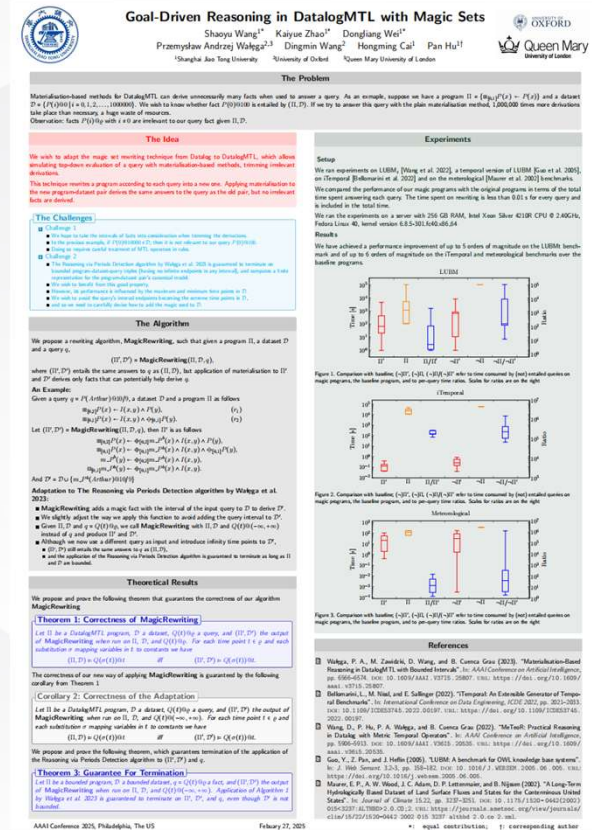
DatalogMTL is a powerful rule-based language for temporal reasoning. Due to its high expressive power and flexible modeling capabilities, it is suitable for a wide range of applications, including tasks from industrial and financial sectors. However, due its high computational complexity, practical reasoning in DatalogMTL is highly challenging. To address this difficulty, we introduce a new reasoning method for DatalogMTL which exploits the magic sets technique—a rewriting approach developed for (non-temporal) Datalog to simulate top-down evaluation with bottom-up reasoning. We have implemented this approach and evaluated it on publicly available benchmarks, showing that the proposed approach significantly and consistently outperformed state-of-the-art reasoning techniques.

a program-dataset pair $(\Pi, \mathcal{D})$ logically entails a given temporal fact. This task was shown to be EXPSpace-complete in combined complexity (Brandt et al. 2018) and PSPACE-complete in data complexity (Wałęga et al. 2019). To decrease computational complexity, various syntactical (Wałęga et al. 2020b) and semantic (Wałęga et al. 2020a) modifications of DatalogMTL have been introduced. DatalogMTL was also extended with stratified negation (Tena Cucala et al. 2021), non-stratified negation (Wałęga et al. 2021, 2024), and with temporal aggregation (Bellomarini, Nissi, and Sallinger 2021).

A sound and complete reasoning approach for DatalogMTL can be obtained using an automata-based construction (Wałęga et al. 2019). Another approach for reasoning is to apply materialisation, that is, successively apply to the

↑论文发表

海报展示 →



03

项目总结与心得





项目总结



项目组成员在整个研究过程中表现出了高度的积极性和责任感。每位成员都充分发挥了自己的专业优势，分工明确，协作紧密。无论是**理论研究**、**算法设计**，还是**系统实现**和**实验评估**，大家都投入了大量的时间和精力，确保了项目的顺利推进。项目最终出色达到了预期目标。



Pan

12-10 07:48:50

我们的论文被AAAI 2025正式录用，祝贺大家！



Royal Raisins

12-10 07:50:17

感谢胡老师一直的大力支持，感谢两位学弟的帮助！



HoriZon

12-10 08:00:03

感谢老师感谢学长 🎉🎉🎉



三三的九

12-10 08:12:01

感谢老师、学长和组长的支持和帮助！ 🙏🎉





1、创新思维

项目成员突破传统思维的束缚，进行大胆的假设与尝试，最终得出简洁但富有创新性的算法成果。

2、创新实践

项目成员将创新思维转化为实际成果。项目初期，项目成员通过教科书阅读、文献调研、沟通交流等方式，从零开始入门一个复杂的领域。在代码原型的实践中，项目成员不仅锻炼了编程能力，还掌握了优化算法的设计思路和实现方法，为今后解决类似问题积累了经验。项目成员完整地参与到科研从选题到论文发表和结题的全过程，学术能力和科研素养得到提升，激发了科研热情。





3、团队协作

项目的顺利完成离不开团队成员的紧密合作。在项目过程中，我们分工明确，相互支持，充分发挥了各自的优势。无论是理论探讨还是技术实现，大家都积极沟通，及时解决问题。这种协作模式不仅提高了工作效率，也让我们深刻体会到团队合作的重要性。

4、未来展望

尽管项目取得了阶段性成果，但我们也认识到系统在扩展性和功能完善性方面仍有提升空间。未来，我们计划进一步优化算法，扩展系统的功能，并探索更多应用场景。此外，我们也希望加强系统的用户体验设计，使其更易于使用和维护。



感谢老师!

上海交通大学

SHANGHAI JIAO TONG UNIVERSITY

饮水思源 爱国荣校